

C++ / OS / Computer Architecture Roadmap (Basic → Job-Ready)

Built for an ECE student aiming at software/embedded/systems roles (SDE, firmware, embedded software, VLSI-adjacent software, driver dev, etc.). Total timeline: ~**7-9 months** at a steady pace (10-15 hrs/week). Compress to 4-5 months if full-time.

How to use this roadmap

- The three tracks (C++, OS, Computer Architecture) are meant to run **in parallel**, not sequentially — they reinforce each other (e.g., you can't really "get" OS without pointers/memory from C++, and Computer Architecture explains *why* OS does what it does).
 - Each phase ends with a **checkpoint** — a mini-project or set of problems that prove you actually absorbed it.
 - "Job level" here means: you can pass a systems/embedded SDE interview (DSA + OS + CN/Arch questions), read and modify real C++ codebases, and reason about performance and memory.
-

Phase 0 — Foundations (2 weeks)

Topic	What to cover
Programming basics	Variables, control flow, functions — skip if you already code
Number systems	Binary, hex, two's complement, floating point representation (IEEE 754)
Linux basics	Shell navigation, <code>gcc</code> / <code>g++</code> , <code>make</code> , basic <code>git</code>

Checkpoint: Compile and run a multi-file C++ program using `g++` and a simple `Makefile`.

Track 1 — C++ (Basic → Advanced → Job-ready)

Stage 1: Core Language (3-4 weeks)

- Syntax, data types, operators, control flow
- Functions: overloading, default args, pass by value/reference/pointer

- Arrays, strings (`std::string`), `std::vector`
- Pointers & references — this is the single most important concept to nail early
- Structs, enums
- Compilation model: preprocessor, compiler, linker; header files vs source files

Checkpoint: Build a small CLI tool (e.g., a text-based inventory manager) using only Stage 1 concepts.

Stage 2: Object-Oriented Programming (3 weeks)

- Classes, constructors/destructors, `this`
- Encapsulation, inheritance, polymorphism (virtual functions, vtables — know *how* they work under the hood)
- Operator overloading
- Abstract classes / interfaces
- RAI (Resource Acquisition Is Initialization) — foundational C++ idiom

Checkpoint: Design a small class hierarchy (e.g., Shape → Circle/Rectangle) with virtual functions and explain the vtable layout on paper.

Stage 3: Memory Management (3 weeks) — *critical for interviews*

- Stack vs heap
- `new`/`delete`, memory leaks, dangling pointers
- Smart pointers: `unique_ptr`, `shared_ptr`, `weak_ptr`
- Rule of 3 / Rule of 5 (copy constructor, copy assignment, move constructor, move assignment, destructor)
- Move semantics, `std::move`, rvalue references

Checkpoint: Implement a custom `unique_ptr`-like RAI wrapper from scratch.

Stage 4: STL & Modern C++ (4 weeks)

- Containers: `vector`, `map`, `unordered_map`, `set`, `deque`, `stack`, `queue`
- Iterators, algorithms (`sort`, `find`, `accumulate`, etc.)
- Templates: function templates, class templates, basic template metaprogramming
- Lambdas, `std::function`
- C++11/14/17/20 features: `auto`, structured bindings, `constexpr`, ranges (17/20 basics)

Checkpoint: Solve 30–40 DSA problems (LeetCode easy/medium) using STL idiomatically.

Stage 5: Concurrency & Advanced Topics (3-4 weeks)

- Threads (`std::thread`), mutexes, locks, condition variables
- Race conditions, deadlocks, atomic operations
- `std::async`, futures/promises (overview)
- Exception handling and exception safety guarantees
- Compilation deep-dive: static vs dynamic linking, undefined behavior, `const`/`constexpr`/`volatile`

Checkpoint: Build a multithreaded producer-consumer queue with proper synchronization.

Stage 6: Job-Ready Practice (ongoing)

- 100-150 DSA problems in C++ (arrays, strings, trees, graphs, DP)
- Read one real open-source C++ codebase (e.g., a small library) to see idiomatic style
- Practice explaining vtables, memory layout, and smart pointer internals out loud (common interview asks)

Resources:

- Book: *C++ Primer* (Lippman) for depth, or *A Tour of C++* (Stroustrup) for speed
 - Site: learncpp.com (excellent free reference, use throughout)
 - Practice: LeetCode, Codeforces (Div 2 A/B) for DSA in C++
-

Track 2 — Operating Systems (Basic → Job-ready)

Stage 1: OS Fundamentals (2 weeks)

- What an OS does: process/resource management, abstraction layer over hardware
- System calls vs library calls
- Kernel mode vs user mode

Stage 2: Processes & Threads (3 weeks)

- Process vs thread, process states, PCB (Process Control Block)
- Context switching — mechanism and cost
- Process creation: `fork()`, `exec()`, `wait()` (do this hands-on in C on Linux)
- Threads: user-level vs kernel-level, `pthread` basics

- Inter-Process Communication: pipes, shared memory, message queues, sockets (overview)

Checkpoint: Write a C program that forks a child process and communicates with it via a pipe.

Stage 3: CPU Scheduling (2 weeks)

- Scheduling goals (throughput, latency, fairness)
- Algorithms: FCFS, SJF, Round Robin, Priority Scheduling, Multilevel Queue
- Context switch overhead, starvation, aging

Checkpoint: Simulate 2-3 scheduling algorithms in C++ given a set of processes with arrival/burst times; compare average waiting/turnaround time.

Stage 4: Synchronization (2-3 weeks) — *heavily tested in interviews*

- Critical section problem
- Mutexes, semaphores, monitors
- Classic problems: Producer-Consumer, Readers-Writers, Dining Philosophers
- Deadlocks: conditions, prevention, avoidance (Banker's Algorithm), detection

Checkpoint: Solve the Producer-Consumer and Dining Philosophers problems using `pthread` mutexes/semaphores in C/C++.

Stage 5: Memory Management (3 weeks)

- Address spaces, logical vs physical addresses
- Paging, segmentation
- Page tables, TLB (Translation Lookaside Buffer)
- Page replacement algorithms: FIFO, LRU, Optimal
- Virtual memory, demand paging, thrashing
- Fragmentation (internal/external)

Checkpoint: Implement LRU and FIFO page replacement in C++ and compare page fault counts on a sample reference string.

Stage 6: File Systems & I/O (2 weeks)

- File system structure, inodes, directory structures
- Disk scheduling algorithms: FCFS, SCAN, C-SCAN, SSTF
- I/O handling basics: polling vs interrupts vs DMA (ties into Computer Architecture track)

Stage 7: Job-Ready Practice (ongoing)

- Standard OS interview question bank (deadlock, paging, scheduling numericals)
- Be able to explain, from memory, what happens from `int main()` to program exit (loader, stack setup, syscalls, exit)

Resources:

- Book: *Operating System Concepts* (Silberschatz, "Dinosaur book") — the standard
 - Course: any solid OS course covering the above (many free university lecture series exist — search current offerings, since good MOOCs change over time)
 - Practice: implement toy versions of scheduler/page-replacement in code, not just theory
-

Track 3 — Computer Architecture (Basic → Job-ready)

Stage 1: Digital Logic Refresher (2 weeks) — *should be easy for ECE*

- Logic gates, combinational vs sequential circuits
- Flip-flops, registers, counters
- Boolean algebra, K-maps (just enough to move fast — you likely know this from ECE coursework)

Stage 2: Instruction Set Architecture (2 weeks)

- CPU registers, ALU, control unit
- Instruction formats, addressing modes
- RISC vs CISC
- Assembly language basics (x86 or ARM/RISC-V — pick one, RISC-V is a great modern choice)

Checkpoint: Write and trace through a small assembly program by hand (register/memory state after each instruction).

Stage 3: Datapath & Pipelining (3 weeks)

- Single-cycle and multi-cycle datapath design
- Pipelining: stages (Fetch, Decode, Execute, Memory, Writeback)
- Hazards: structural, data, control — and how they're resolved (forwarding, stalling, branch prediction)

Checkpoint: Trace a sequence of instructions through a 5-stage pipeline and identify hazards on paper.

Stage 4: Memory Hierarchy (3 weeks) — *overlaps heavily with OS Track, do together*

- Cache basics: direct-mapped, set-associative, fully associative
- Cache hit/miss, write policies (write-through/write-back)
- Multi-level caches (L1/L2/L3)
- Locality of reference (temporal/spatial) — the "why" behind caching
- Virtual memory tie-in (already covered in OS track — connect the dots here)

Checkpoint: Calculate cache hit/miss rate and average access time for a given cache configuration and access pattern.

Stage 5: I/O, Interrupts & Parallelism (2–3 weeks)

- Interrupts vs polling, interrupt handling flow
- DMA (Direct Memory Access)
- Buses, I/O interfacing
- Basics of multicore architecture, cache coherence (MESI protocol overview)
- Amdahl's Law

Stage 6: Job-Ready Practice (ongoing)

- Standard computer architecture interview questions (cache numericals, pipelining hazards, AMAT calculations)
- Understand how a C++ program's performance connects to cache behavior (why row-major array traversal is faster than column-major, etc.) — great way to unify all three tracks

Resources:

- Book: *Computer Organization and Design* (Patterson & Hennessy) — the standard, and RISC-V edition is very approachable
 - ECE coursework you've already done (COA/Digital Logic) — leverage it, don't restart from zero here
-

Suggested Timeline (parallel tracks)

Month	C++	OS	Computer Architecture
1	Core Language + OOP	Fundamentals + Processes	Digital Logic + ISA
2	Memory Management	Scheduling	Datapath & Pipelining
3	STL & Modern C++	Synchronization	Memory Hierarchy
4	Concurrency	Memory Management	I/O, Interrupts, Parallelism
5	STL & Modern C++	Memory Management	I/O, Interrupts, Parallelism
6	Concurrency	File Systems	Job-ready review
7-9	DSA practice + projects (all tracks) — mock interviews, revision, resume-ready projects		

Capstone Projects (pick 2-3 to build real credibility)

1. **A simple shell** (C/C++) — implements `fork()`/`exec()`/`wait()`, pipes, redirection. Ties C++ + OS together directly.
2. **A multithreaded memory allocator** — custom `malloc()`/`free()` implementation with a free list. Ties C++ + OS memory management together.
3. **A cache simulator** — takes a memory trace and simulates hit/miss rates for different cache configurations. Ties Architecture + C++ together.
4. **A CPU scheduler simulator** with a small visualizer (Gantt chart output). Ties OS + C++ together.
5. **A basic key-value store** with an LRU eviction policy (great DSA + OS + systems-design crossover project, and a strong resume item).

Interview-Prep Checklist (final stretch)

- ☐ 150+ DSA problems solved in C++ (comfortable with arrays, strings, trees, graphs, DP, sliding window)
 - ☐ Can explain vtables, RAII, smart pointers, move semantics without notes
 - ☐ Can solve deadlock/scheduling/paging numericals under time pressure
 - ☐ Can compute cache hit rate / AMAT and explain hazards in a pipeline
 - ☐ Built at least 2 capstone projects and can explain design tradeoffs
 - ☐ Comfortable with `git`, basic Linux CLI, and reading a `Makefile`/`CMakeLists.txt`
 - ☐ Mock interviews done (peer or platform) covering DSA + CS fundamentals
-

Note: Since some course/MOOC recommendations shift over time, it's worth doing a quick search for currently well-reviewed free OS and computer architecture courses when you're ready to start — the core topic list above stays valid regardless of which course you pick.